

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа 3

Выполнил:

Захматов Юрий

Группа:

K3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

- реализовать Dockerfile для каждого сервиса;
- написать общий docker-compose.yml;
- настроить сетевое взаимодействие между сервисами.

Ход работы

1. Создание Dockerfile для каждого сервиса

Для каждого микросервиса был создан Dockerfile, который описывает процесс сборки и запуска приложения. Использован многостадийный подход: на первом этапе происходит сборка JAR-файла с помощью Gradle wrapper, на втором — запуск приложения на легковесном JRE образе.

Все Dockerfile унифицированы — различаются только названиями сервисов и портами.

2. Написание docker-compose.yml

Создан общий файл docker-compose.yml, который описывает все сервисы:

- **Инфраструктурные сервисы:** Eureka Server (порт 8081), API Gateway (порт 8080), Kafka
- **Базы данных:** 6 отдельных контейнеров Postgres (auth_db, property_db, booking_db, payment_db, chat_db, review_db)
- **Микросервисы:** Auth, Property, Booking, Payment, Chat, Reviews

3. Настройка сетевого взаимодействия

Docker network: Все контейнеры помещены в общую сеть microservices-net, что позволяет им обращаться друг к другу по имени контейнера (например, auth-service или postgres-auth).

Healthcheck и depends_on: Для Postgres настроены healthcheck'и (проверка готовности через pg_isready). Микросервисы используют depends_on с условием service_healthy, что гарантирует запуск только после готовности БД.

Переменные окружения: В каждом микросервисе через переменные окружения задаются адреса Eureka (eureka-server:8081), Kafka (kafka:9092), Postgres (postgres-auth:5432 и т.д.).

4. Проброс портов

- **Только Gateway** имеет проброшенный порт на хост (8080) — единая точка входа
- **Eureka** (8761) проброшен для мониторинга
- **Базы данных** порты не проброшены — они доступны только внутри Docker-сети для безопасности
- **Микросервисы** порты не проброшены — они видят друг друга через сеть Docker, но недоступны снаружи напрямую

Вывод

Контейнеризация позволила полностью стандартизировать окружение для всех микросервисов. Разработчик может запустить всю систему одной командой без необходимости вручную поднимать БД, Kafka и сервисы. Деплой на сервер также сводится к выполнению `docker-compose up`. Сетевое взаимодействие через внутреннюю сеть Docker обеспечивает безопасность и удобство общения между сервисами.